Docker Submission Guide

Zalo AI Challenge 2026 – DataMate

1. Overview 2
2. Test machine 2
3. Input and output 3
   3.1. **/data** 3
   3.2. **/result/submission.jsonl** 3
   3.3. **/result/logs/** 3
4. Folder layout in the image 4
5. **predict.sh** rules 5
6. Reproducibility 5
7. Run time 5
8. README and training code 5
9. Build the image 6
10. Check before you submit 8
11. Submit the image 9

1. Overview

Submit one Docker image containing the complete inference solution, training code/data and verification artifacts. Inference must work offline. Only open-source training-only base checkpoints may use public links.

Submit and freeze your image before private test release. Then run it and upload **submission.jsonl** for the provisional leaderboard. The organizers' re-run determines your final score and ranking (Rule 8).

The image must contain:

Item	Notes
Inference code	Code and dependencies to read **/data** and write **/result**, fully offline.
Training code	Training/fine-tuning source, configs, dependencies, logs and run instructions (section 8).
Training data pipeline	Generation/preprocessing code, training/validation data, logs, verification checkpoints and a manifest of local paths and checksums (section 8).
Model weights	All weights, tokenizers and configs needed for inference, including both base model and adapter when used.
predict.sh	Entry point: **/data** → **/result/submission.jsonl** and **/result/logs/**.
README.md	Approach, model declarations, and run instructions (section 8).

2. Test machine

Item	Value
GPU	2 × NVIDIA RTX 5090, 32 GB VRAM each (64 GB total)
RAM	512 GB
CUDA	CUDA ≥12.8; matching PyTorch/vLLM build with Blackwell support.
Base image	Recommended: vllm/vllm-openai:v0.10.1.1 (tested with PyTorch 2.7.1+cu128 and vLLM 0.10.1.1). Alternatives must support RTX 5090 / compute capability 12.0.
Network	Disabled during inference: no APIs or downloads.
/data	Read-only. Write temporary files elsewhere, e.g. **/tmp**.
Time limit	6 hours for the complete private test set, including model loading (Rule 7).
Models	All inference models combined: ≤15B parameters, including every MoE parameter, even if loaded separately. Includes language, embedding, reranking, and classification models. Quantization allowed (Rule 2).

Item	Value
Max image size	100 GB for the uncompressed docker save .tar, including all image layers.

3. Input and output

3.1. /data

Each test split contains the business files, questions, and shared rulebook/calendar shown below.

```
/data/
├── data/<business_id>/
│   ├── *.csv | *.xlsx | *.sqlite
│   └── README.md
├── rulebook.md              # split-specific rules and vocabulary
├── calendar.csv             # shared solar-lunar calendar
└── questions.jsonl          # id, business_id, question, as_of_date
```

3.2. /result/submission.jsonl

One JSON object per line: id, type and answer for value types (number, percent, string, date, boolean, list, table). unanswerable may include reason; abstain needs no answer. Missing questions count as abstain.

Write an answer immediately. If the run stops early or exceeds 6 hours, only saved answers are scored; the rest count as abstain.

3.3. /result/logs/

Write one log record per question under **/result/logs/**. Any format is accepted. Required fields:

- id: question ID.
- answer: final answer.
- elapsed_ms: non-negative integer wall-clock milliseconds from question start to final answer, including data reads, reasoning and code execution. With batching, measure from when the question enters its batch. Shared model loading is separate but counts toward 6 hours.
- thinking: the model's reasoning for this question, recorded as generated (e.g., the text inside <think>…</think> or vLLM's reasoning_content). Do not summarize or truncate it. If several model calls were made for the question, record a list of strings in call order. Use null if no reasoning was generated (e.g., thinking mode disabled or no model call).

For questions solved by code, also record:

- code: the actual generated code that was executed. For predefined Python, record the source file/function and actual inputs. Generated code may call packaged helper functions; log the actual call.
- output: the computed return value or execution output. Printing to stdout is not required.

If no code is executed, code and output may be omitted or null. Code generation is optional; logs must reflect the actual computation.

Two alternative log records for the same question are shown below, expanded for readability. Write one record per question, on one line, in the actual file.

Example 1. Model-generated code calling a packaged helper.

```
 {
  "id": "q_01234",
  "answer": {
    "type": "number",
    "answer": 48250000
  },
  "elapsed_ms": 8421,
  "thinking": "The question asks for biz_017 revenue month-to-date as of 2026-08-27, so
the range is 2026-08-01 to 2026-08-27. sales.csv has date and amount columns; call
compute_revenue over that range.",
  "code": "from src.metrics import compute_revenue\nresult =
compute_revenue('/data/data/biz_017', start='2026-08-01', end='2026-08-27')",
  "output": 48250000
}
```

Example 2. Calling a packaged function.

```
 {
  "id": "q_0123",
  "answer": {
    "type": "number",
    "answer": 48250000
  },
  "elapsed_ms": 125,
  "thinking": null,
  "code": "src/metrics.py::compute_revenue",
  "inputs": {
    "data_dir": "/data/data/biz_017",
    "start": "2026-08-01",
    "end": "2026-08-27"
  },
  "output": 48250000
}
```

In Example 2, inputs are a supporting field for the actual arguments, and thinking is null because no model reasoning was generated. Referenced source files/functions must be inside the Docker image; output may be a returned value without stdout.

4. Folder layout in the image

Suggested internal layout. **/code/predict.sh** and the section 3 input/output paths are fixed. Document other folder locations in **/code/README.md**.

```
/code/
├── predict.sh               # entry point
├── README.md
├── requirements.txt
├── src/                     # inference code
├── models/                  # model weights
└── training_code/
    ├── README.md
    ├── requirements.txt     # pinned training dependencies
    ├── train.sh             # training entry point, if trained
    ├── configs/             # training and generation settings
    ├── src/                 # training source code, if trained
    ├── data_generation/     # generation and preprocessing scripts
    ├── data/                # train/validation data
    ├── logs/                # training logs
    ├── checkpoints/         # verification checkpoints, if needed
    └── data_manifest.json   # local paths, SHA-256; base model public URLs
```

Package all training data, code, dependencies and verification evidence. Only open-source training-only base checkpoints may be linked (section 8). Remove redundant files; retain optimizer state only if verification needs it.

For inference, include both the base model and adapter, or a merged checkpoint. A public base-model link does not replace required inference weights.

5. predict.sh rules

1. Run bash **/code/predict.sh** without arguments.
2. Read **/data**; write **submission.jsonl** and per-question logs under **/result**. Create missing output directories.
3. Set fixed defaults for model, seed, temperature, batch size, and other settings. The organizers will not modify them.
4. No manual intervention, hard-coded answers or logic keyed on question/business IDs.
5. Agents may generate and execute code inside the container (optional; Rule 5).

6. Reproducibility

Reproducibility requires |PA(uploaded) − PA(re-run)| ≤ 0.03. Use fixed seeds or greedy decoding. The re-run score is final (Rule 8).

Case	What happens
|PA(uploaded) − PA(re-run)| ≤ 0.03	The re-run score is your final score.
Difference > 0.03	The organizers review the logs and may re-run the image or ask your team for more information.
Difference cannot be explained	The entry is disqualified.

GPU batching may change outputs between runs, even with greedy decoding. Before you submit, run your image twice on the same input and compare the two submission.jsonl files, or compute PA on the training set (it has answers.jsonl). Rule 8 compares your uploaded private-test PA with the organizers' re-run.

7. Run time

- Process the complete private test within 6 wall-clock hours on the reference machine, including loading and initialization.
- Reference only: Qwen3-14B initialization took about 2 minutes (128,155 ms), including compilation and CUDA graphs.
- At the time limit, the organizers stop the container; unsaved answers count as abstain.
- Tie-breaks: lower total inference time in the organizers' re-run, then earlier submission.

8. README and training code

/code/README.md must include:

- Approach: reading data, generating/executing code, and choosing answers.
- Models: name, public source, fixed revision, original release date, license, and parameter count. Original weights must predate 26 October 2026 with a license permitting use. Fine-tuning is allowed; apply the 15B total in section 2.
- Data: sources/versions, generation/preprocessing method and train/validation sample counts.
- Allowed: provided training data, self-generated data and public datasets/models. Never train on public/private test data or collect test answers (Rule 6).

/code/training_code/ must support verification and reproduction. Include:

- Code/environment: training and data-generation/preprocessing source, configs, prompts/templates, pinned dependencies and seeds. Install dependencies or bundle an offline environment.
- Data/evidence: all train/validation data, logs, run configs and verification checkpoints inside the image. Save prompts/responses for API-generated or non-reproducible data.
- README: exact preparation/training/export commands, hardware, runtime, generator model/settings, final checkpoint/adapter and any nondeterministic steps.
- **data_manifest.json**: local paths, sources/versions, file sizes, SHA-256 checksums and dataset counts/splits.

For omitted open-source training-only base checkpoints, provide public URLs, fixed revisions, licenses, required files and SHA-256 checksums. All inference assets stay inside the image.

The organizers may reproduce preparation/training separately from the 6-hour inference limit. State explicitly if no training or data generation was performed.

9. Build the image

Build for linux/amd64 with all required inference/training assets. Make sure .dockerignore does not exclude training data. Exclude only unused files and build caches.

Sample stack: Docker 24.0.5 with BuildKit.

1. Install Docker/BuildKit and NVIDIA Container Toolkit. RTX 5090 requires a compatible R570-or-newer driver; follow the official guides below.

Docker: https://docs.docker.com/engine/install/
NVIDIA Container Toolkit: https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/install-guide.html

2. Pull the base image: docker pull vllm/vllm-openai:v0.10.1.1

```
user@gpu5090:~/qwen_smoke$ docker --version
Docker version 24.0.5, build ced0996
user@gpu5090:~/qwen_smoke$ nvidia-smi --query-gpu=index,name,driver_version --format=csv
index, name, driver_version
0, NVIDIA GeForce RTX 5090, 570.86.10
1, NVIDIA GeForce RTX 5090, 570.86.10
2, NVIDIA GeForce RTX 5090, 570.86.10
3, NVIDIA GeForce RTX 5090, 570.86.10
user@gpu5090:~/qwen_smoke$ docker run --rm --gpus all --entrypoint "" vllm/vllm-openai:v0.10.1.1 python3 -c "
  import torch, vllm
  print('torch', torch.__version__, 'cuda', torch.version.cuda, '| vllm', vllm.__version__)
  for i in range(torch.cuda.device_count()): print(i, torch.cuda.get_device_name(i), torch.cuda.get_device_capability(i))
  print('p2p 0<->1:', torch.cuda.can_device_access_peer(0, 1))"
torch 2.7.1+cu128 cuda 12.8 | vllm 0.10.1.1
0 NVIDIA GeForce RTX 5090 (12, 0)
1 NVIDIA GeForce RTX 5090 (12, 0)
2 NVIDIA GeForce RTX 5090 (12, 0)
3 NVIDIA GeForce RTX 5090 (12, 0)
p2p 0<->1: False
```

Figure 1. Sample environment: four-GPU test host; scoring uses two GPUs.

3. Copy code, inference weights, training data and verification materials into **/code**. Include dependencies and the artifact manifest.
4. Install libraries in the Dockerfile. For the recommended two-GPU stack:
   - Reset ENTRYPOINT [] so that a plain docker run starts predict.sh, not the vLLM server. The scoring command also sets --entrypoint bash (section 10).
   - The sample fixed a two-GPU initialization hang by upgrading NCCL 2.26.2 to nvidia-nccl-cu12==2.27.7. Validate this workaround with your stack.
   - The tested GPU pair lacked P2P access; the sample sets NCCL_P2P_DISABLE=1. vLLM disables custom all-reduce when P2P is unavailable.
   - Set HF_HUB_OFFLINE=1 and TRANSFORMERS_OFFLINE=1; load packaged local models.
5. Build and inspect the image. Use **zaic2026_datamate_<team_name>:final**, replacing <team_name> with a lowercase team identifier.

```
docker build --platform linux/amd64 \
  -t zaic2026_datamate_<team_name>:final .
docker image inspect \
  -f '{{.Os}}/{{.Architecture}} {{.Size}}' \
  zaic2026_datamate_<team_name>:final
docker run --rm --network none --entrypoint "" \
  zaic2026_datamate_<team_name>:final find /code -maxdepth 2
```

```
user@gpu5090:~/qwen_smoke$ docker build --platform linux/amd64 --target submit -t datamate_qwen14b:v1 .
[+] Building 564.1s (12/12) FINISHED                                    docker:default
 => [internal] load build definition from Dockerfile                             0.0s
 => [internal] load .dockerignore                                                0.0s
 => [internal] load metadata for docker.io/vllm/vllm-openai:v0.10.1.1            0.0s
 => [app 1/6] FROM docker.io/vllm/vllm-openai:v0.10.1.1                          0.0s
 => [internal] load build context                                              260.1s
 => => transferring context: 29.56GB                                           260.1s
 => [app 3/6] COPY code/requirements.txt /code/requirements.txt                 5.3s
 => [app 4/6] RUN pip install -r /code/requirements.txt  && pip show nvidia-nccl-cu12  34.3s
 => [app 5/6] COPY code/ /code/                                                  0.0s
 => [app 6/6] RUN chmod +x /code/predict.sh && mkdir -p /code/models             0.1s
 => [submit 1/1] COPY weights/ /code/models/                                   114.1s
 => exporting to image                                                         150.0s
 => => naming to docker.io/library/datamate_qwen14b:v1                           0.0s
user@gpu5090:~/qwen_smoke$ docker images datamate_qwen14b:v1
REPOSITORY         TAG       IMAGE ID       CREATED         SIZE
datamate_qwen14b   v1        d4d7521cb803   2 minutes ago   50.5GB
```

Figure 2. Building the sample image with Qwen3-14B weights. The sample used a two-stage Dockerfile (--target submit); with the single-stage reference Dockerfile below, omit --target.

```
user@gpu5090:~/qwen_smoke$ docker image inspect -f '{{.Os}}/{{.Architecture}} {{.Size}}' datamate_qwen14b:v1
linux/amd64 50489441971
user@gpu5090:~/qwen_smoke$ docker run --rm --network none --entrypoint "" datamate_qwen14b:v1 bash -c \
      'cd /code && find . -maxdepth 3 -not -path "./models/*/*" | sort; du -sh /code/models/*'
.
./README.md
./models
./models/Qwen3-14B
./predict.sh
./requirements.txt
./src
./src/predict.py
./training_code
./training_code/README.md
./training_code/data_manifest.json
28G     /code/models/Qwen3-14B
```

Figure 3. Checking image platform, size and folder layout.

Reference Dockerfile with the sample NCCL workaround. Separate COPY steps to keep weights cached when source code changes.

```
FROM vllm/vllm-openai:v0.10.1.1
ENTRYPOINT []
ENV HF_HUB_OFFLINE=1 TRANSFORMERS_OFFLINE=1 \
    PIP_NO_CACHE_DIR=1 NCCL_P2P_DISABLE=1
WORKDIR /code
COPY requirements.txt /code/requirements.txt
RUN pip install --no-cache-dir -r /code/requirements.txt
RUN pip install --no-cache-dir --no-deps nvidia-nccl-cu12==2.27.7
COPY models/ /code/models/
COPY training_code/ /code/training_code/
COPY predict.sh README.md /code/
COPY src/ /code/src/
RUN chmod +x /code/predict.sh
CMD ["bash", "/code/predict.sh"]
```

Install training dependencies or bundle an offline training environment under **training_code/**. Test the finished image with section 10's command.

Disk planning: the sample used about 57 GB for image/build cache and another 50 GB temporarily during export on Docker 24. Actual needs vary; also allow space for training data and the archive.

10. Check before you submit

Test with the scoring command: two GPUs, no network, read-only input, and 16 GiB shared memory.

The organizers provide no privileged mode, Docker socket, host credentials, or host services.

```
docker run --rm --gpus '"device=0,1"' --network none --shm-size=16g \
  --entrypoint bash \
  -v /path/to/public_test:/data:ro \
  -v /path/to/output:/result \
  zaic2026_datamate_<team_name>:final /code/predict.sh
```

The sample exceeded the default 64 MiB **/dev/shm** during NCCL allocation. --shm-size=16g resolved that capacity issue without host IPC. The scoring command sets --entrypoint bash, so the image's own ENTRYPOINT is not used.

```
user@gpu5090:~/qwen_smoke$ docker run --rm --gpus '"device=2,3"' --network none -e GPU_MEM_UTIL=0.85 \
      -v $PWD/data/public_test:/data:ro -v $PWD/out/submit_10:/result \
      datamate_qwen14b:v1 bash /code/predict.sh --limit 10
[predict.sh] gpus=2 shm_kb=65536 NCCL_SHM_DISABLE=1
[info] model=/code/models/Qwen3-14B/ tp=2 câu=10 còn=10
[info] rulebook=18799 ký tự
(VllmWorker TP0 pid=289) INFO 11-02 03:20:17 [default_loader.py:262] Loading weights took 27.81 seconds
(VllmWorker TP0 pid=289) INFO 11-02 03:21:20 [monitor.py:34] torch.compile takes 41.25 s in total
...
(VllmWorker TP0 pid=289) ERROR 11-02 03:21:20 [multiproc_executor.py:596] torch.distributed.DistBackendError: NCCL error in: /pytor
ch/torch/csrc/distributed/c10d/NCCLUtils.cpp:77, unhandled system error (run with NCCL_DEBUG=INFO for details), NCCL version 2.27.7
(VllmWorker TP0 pid=289) ERROR 11-02 03:21:20 [multiproc_executor.py:596] Error while creating shared memory segment /dev/shm/nccl-
xLah0q (size 33030528), error: No space left on device (28)
```

Figure 4. Sample NCCL allocation failure with default 64 MiB shared memory. Setting NCCL_SHM_DISABLE=1 does not fix it; use --shm-size. --limit 10 is a sample-only option; the organizers run predict.sh without arguments.

```
user@gpu5090:~/qwen_smoke$ docker run --rm --gpus '"device=2,3"' --network none --shm-size=16g -e GPU_MEM_UTIL=0.80 \
      -v $PWD/data/public_test:/data:ro -v $PWD/out/submit_10:/result \
      datamate_qwen14b:v1 bash /code/predict.sh --limit 10
[predict.sh] gpus=2 shm_kb=16777216 NCCL_SHM_DISABLE=0
[predict.sh] start 2026-11-02T03:35:36-08:00
[info] model=/code/models/Qwen3-14B/ tp=2 câu=10 còn=10
[info] rulebook=18799 ký tự
(EngineCore_0 pid=283) INFO 11-02 03:37:40 [kv_cache_utils.py:849] GPU KV cache size: 134,048 tokens
(VllmWorker TP0 pid=289) INFO 11-02 03:37:45 [gpu_model_runner.py:2708] Graph capturing finished in 6 secs, took 1.66 GiB
[time] load_model_ms=128155
[ans] q_009187cd2960 lần 2 number 250
[ans] q_00bf8df3ea0b lần 2 number 0
... (8 câu nữa)
[time] chunk 1: 10 câu, 42622ms/câu
[info] thống kê type: {'number': 4, 'abstain': 5, 'list': 1}
[time] total_ms=554385
[predict.sh] done  2026-11-02T03:44:54-08:00
user@gpu5090:~/qwen_smoke$ head -2 out/submit_10/submission.jsonl
{"id": "q_009187cd2960", "type": "number", "answer": 250}
{"id": "q_00bf8df3ea0b", "type": "number", "answer": 0}
```

Figure 5. Ten-question smoke test completed with --shm-size=16g. --limit 10 is a sample-only option; the organizers run predict.sh without arguments. This does not establish full-test accuracy, determinism or runtime.

Check that:

- It runs to the end with no errors and no network.
- **submission.jsonl** covers every question in the published format.
- Each question has a log record with id, answer, elapsed_ms and thinking (null if no reasoning was generated).
- Estimated private-test time fits within 6 hours: model loading + (average time per question on a public or training run × 500).
- Two runs give the same answers, or their PA on the training set differs by at most 0.03 (section 6).
- Inference works offline; all training data, code, and verification evidence are packaged.
- Export one linux/amd64 image/tag; its uncompressed .tar size is at most 100 GB (section 2).

11. Submit the image

1. Export only the final linux/amd64 image/tag and compress it for upload. Run in Bash and replace <team_name> with your lowercase team identifier. Run set -o pipefail first: without it, a failed docker save still leaves a small .tar.gz and the command reports success. Then measure the uncompressed .tar size from the archive:

```
set -o pipefail
docker save zaic2026_datamate_<team_name>:final | gzip >
zaic2026_datamate_<team_name>.tar.gz
# uncompressed .tar size in bytes:
gzip -dc zaic2026_datamate_<team_name>.tar.gz | wc -c
```

Model weights compress poorly: the sample image went from 50.6 GB as .tar to 34.4 GB as .tar.gz. pigz -p 8 (and pigz -dc) is much faster than gzip and produces a standard .tar.gz.

2. On the build machine, before uploading, write the MD5 checksum of the .tar.gz to a file (on macOS, use md5 -r instead of md5sum):

```
md5sum zaic2026_datamate_<team_name>.tar.gz > zaic2026_datamate_<team_name>.tar.gz.md5
```



Figure 6. On the build machine: the uncompressed .tar size and the MD5 file sent with the upload. On a machine that downloaded the archive: md5sum -c compares that copy with the MD5 from the build machine.

3. Upload the .tar.gz and send its link, the .md5 file (or its one line) and the uncompressed .tar size in bytes to the organizers. The organizers check their copy with **md5sum -c zaic2026_datamate_<team_name>.tar.gz.md5: OK** means the archive arrived intact, FAILED means it must be uploaded again.
4. Do not rename the archive after creating the .md5 file. Before sending, download the archive from your link on another machine and run the same check, so you know the link works, and the file is intact.
5. The artifact manifest must be inside the image; any external model reference is limited to an eligible open-source training-only base checkpoint.

After the solution deadline, you may not change the image, code, configs, models, training data, generation artifacts or the declared public base-checkpoint revisions and checksums.